Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ（メソッド)」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（**View**)、データの処理（**Model**)、そしてユーザの入力と画面表示の制御（**Controller**）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。

Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ（メソッド)」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（**View**）、データの処理（**Model**）、そしてユーザの入力と画面表示の制御（**Controller**）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。

Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ（メソッド）」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（**View**）、データの処理（**Model**）、そしてユーザの入力と画面表示の制御（**Controller**）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。

Spring の特徴

1. DI(依存性注入)

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ(メソッド)」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC(Model-View-Controller)モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示(**View**)、データの処理(**Model**)、そしてユーザの入力と画面表示の制御(**Controller**)といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。